Lab 4.2: Adding Open File Dialogs and a Rich Edit View

In this lab, you will use a **Common Open File** dialog box and a rich edit view in an application.

To see a demonstration of the solution for this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 4.2 Demonstration.")

Estimated time to complete this lab: **60 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch04\Lab4.2.

The solution code for this lab is located in the folder \Labs\Ch04\Lab4.2.

Objectives

After completing this lab, you will be able to:

- Modify the base classes of views and documents.
- Use a **Common Dialogs Library Open** dialog box.
- Use a rich edit view in an application.
- Process a dropped file on the edit control.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter:

Exercise 1: Modifying the Base Classes of Views and Documents

In this exercise, you will use the text editors of Visual C++ to modify the **Document** and **View** classes to inherit from their respective rich edit classes.

Exercise 2: Handling the Common Open File Dialog Box

In this exercise, you will add a handler for the **Open File** dialog box in the **ShowDiff** application.

Exercise 3: Using the Rich Edit Views in the Application

In this exercise, you will use the rich edit views to display the file that the user chooses in the File dialog box. You will create a function that serializes the text data into the rich edit views, and drive that function from the **OnFileOpen** handler.

Exercise 4: Managing Drag-and-Drop Editing

In this exercise, you will modify the behavior of **CRichEdit** so that it will open a file that is dragged from Windows Explorer to the window pane of the running application.

122 Exercise 1: Modifying the Base Classes of Views and Documents

The code that forms the starting point for this exercise is in \Labs\Ch04\Lab4.2\Ex01.

In this exercise, you will use the text editors of Visual C++ to modify the **Document** and **View** classes to inherit from their respective rich edit classes.

▶ **Replace CDocument with CRichEditDoc**

☐ 1. Open the file DiffDoc.cpp.

☐ 2. Replace all instances of **CDocument** with **CRichEditDoc**.

☐ 3. Save DiffDoc.cpp.

☐ 4. Repeat this procedure with the file DiffDoc.h.

▶ **Replace CView with CRichEditView**

☐ 1. Open DiffView.cpp.

☐ 2. Replace all instances of **CView** with **CRichEditView**.

☐ 3. Save DiffView.cpp.

☐ 4. Repeat this procedure with DiffView.h.

▶ **Add the #includes that support rich edit classes to the file Stdafx.h**

☐ 1. Open Stdafx.h.

☐ 2. The **CRichEditDoc** class inherits from **COleDocument**, **COleLinkingDoc**, and **COleServerDoc**. The definitions for these classes and their support functions need to be included before the rich edit definitions. To the end of Stdafx.h, add:

```
#include <afxole.h>
#include <afxodlgs.h>
```

☐ 3. The definitions for rich edit control are in the file Afxrich.h. To the end of Stdafx.h, after the file Afxodlgs.h, add:

```
#include <afxrich.h>
```

☐ 4. Save Stdafx.h.

Notes

▶ Add the support for CRichEditCntrItem

Derived classes from **CRichEditDoc** need to provide a pointer to a control item (the class that contains the rich edit control itself). This simple function is implemented in DiffDoc.h.

☐ 1. Open DiffDoc.h.

☐ 2. Add this code to the overrides section (after the //}AFX_VIRTUAL tag):

```
virtual CRichEditCntrItem* CreateClientItem(REOBJECT* preo) const;
```

☐ 3. Save DiffDoc.h.

☐ 4. Open DiffDoc.cpp.

☐ 5. Add this code right after the destructor (**CDiffDoc::~CDiffDoc**):

```
CRichEditCntrItem* CDiffDoc::CreateClientItem(REOBJECT* preo) const
{
    return new CRichEditCntrItem();
}
```

☐ 6. Save DiffDoc.cpp and build the project.

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.2\Ex01\Solution.

Notes

122 Exercise 2: Handling the Common Open File Dialog Box

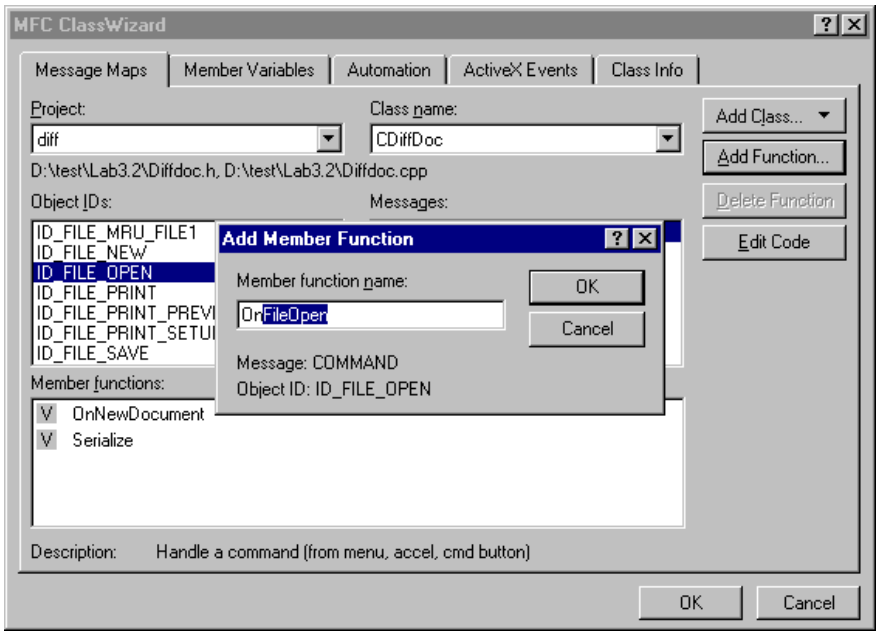
Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch04\Lab4.2\Ex02.

In this exercise, you will add a handler for the **Open File** dialog box to the ShowDiff application.

▶ Add an OnFileOpen handler for CDiffDoc

☐ 1. On the **View** menu, click **ClassWizard** or press CTRL+W. Click the **Message Maps** tab.

☐ 2. Select the **CDiffDoc** class, the ID_FILE_OPEN menu object ID, and the COMMAND message, and click **Add Function**, as shown in this figure.



☐ 3. Accept the default **OnFileOpen** function name in the **Add Member Function** dialog box, and click **OK**.

☐ 4. In ClassWizard, click **OK** to create the function.

▶ Declare protected file name members

- Right-click **CDiffDoc** in ClassView and add these two protected variables:

```
CString m_File1
CString m_File2
```

Notes

▶ Complete the OnFileOpen handler

☐ 1. In the **OnFileOpen** handler of the file DiffDoc.cpp, define the filter for the **File** dialog box. In this example, you will filter for C++ source files (*.cpp and *.h) and for all files:

```
static char BASED_CODE szFilter[] =
    "All Files (*.*)|*.*|C++ Files (*.cpp, *.h)"\
    "|*.cpp;*.h||";
```

☐ 2. Construct the **File** dialog box as an open dialog box with this filter:

```
CFileDialog dlg(TRUE,NULL,NULL,NULL,szFilter);
```

☐ 3. Show the **File** dialog box as a modal dialog box, and check to see whether **OK** was clicked.

☐ 4. When the user clicks **OK**, assign the path to m_File1 and m_File2.

☐ 5. Save DiffDoc.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CDiffDoc::OnFileOpen()
{

    static char BASED_CODE szFilter[] =
        "All Files (*.*)|*.*|C++ Files (*.cpp, *.h)| "\
        "*.cpp;*.h||";

    CFileDialog dlg(TRUE,NULL,NULL,NULL,szFilter);

    if (dlg.DoModal() == IDOK)
    {
        m_File1 = m_File2 = dlg.GetPathName();
    }
}
```

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.2\Ex02\Solution.

Notes

122 Exercise 3: Using the Rich Edit Views in the Application

Continue with the files you created in Exercise 2 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch04\Lab4.2\Ex03.

In this exercise, you will use the rich edit views to display the file that the user chooses in the **File** dialog box. You will create a function that serializes text data into the rich edit views, and drive that function from the **OnFileOpen** handler.

▶ **Get the application and the main frame**

☐ 1. Open the file Diff.h.

☐ 2. Add a forward reference to **CMainFrame** before the definition of **CDiffApp** as follows:

```
class CMainFrame;
```

☐ 3. Define a public method to return a pointer to the application as follows:

```
public:
static CDiffApp * GetApp()
{
    return (CDiffApp *)AfxGetApp();
}
```

☐ 4. Define a public method to a pointer to the main frame as follows:

```
CMainFrame * GetMainFrame()
{
    return (CMainFrame *)m_pMainWnd;
}
```

Note that the purpose of steps 3 and 4 is to simplify writing the code for the rest of this lab.

☐ 5. Save Diff.h.

▶ **Initialize the file members and set the views to text only**

☐ 1. Open the file DiffDoc.cpp and scroll to **CDiffDoc::CDiffDoc**.

☐ 2. Initialize m_File1 and m_File2 as empty strings as follows:

```
m_File1 = _T("");
m_File2 = _T("");
```

☐ 3. The m_bRTF member of **CRichEditDoc** defines whether the streams should store formatting. Because you will only be dealing with text, set this member to **FALSE** as follows:

```
m_bRTF  = FALSE;
```

☐ 4. Save DiffDoc.cpp.

Notes

▶ **Add the function to display the files in the views**

☐ 1. Right-click **CDiffDoc** in ClassView and add the following public function:

```
void RunComparison(LPCSTR lpszFile1, LPCSTR lpszFile2)
```

☐ 2. Open DiffDoc.cpp. Include DiffView.h, the file Splitter.h, and the file MainFrm.h after the statement that includes DiffDoc.h as follows:

```
#include "diffdoc.h"
#include "diffview.h"
#include "splitter.h"
#include "mainfrm.h"
```

▶ **Write the code for the Run Comparison function**

☐ 1. Get the splitter:

```
CMainFrame * pFrame = CDiffApp::GetApp()->GetMainFrame();
CSplitter * pSplitter = pFrame->GetSplitter();
```

☐ 2. Get the first pane as follows:

```
CDiffView * pView;
pView = (CDiffView *)pSplitter->GetPane(0,0);
```

☐ 3. Stream the first file into the first pane as follows:

```
CFile   file(lpszFile1, CFile::modeRead);
CArchive ar(&file, CArchive::load);
pView->Serialize(ar);
```

☐ 4. Repeat the procedure for the second pane.

☐ 5. Mark the document as “clean” so as not to save it:

```
SetModifiedFlag(FALSE);
```

☐ 6. Save DiffDoc.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CDiffDoc::RunComparison (LPCSTR lpszFile1,
                              LPCSTR lpszFile2)
{
    CMainFrame * pFrame = CDiffApp::GetApp()->GetMainFrame();

    if(pFrame)
    {
        CSplitter * pSplitter = pFrame->GetSplitter();

        if(pSplitter)
        {
            CDiffView * pView;

            pView = (CDiffView *)pSplitter->GetPane(0,0);

            if(pView)
            {
                CFile   file(lpszFile1, CFile::modeRead);
                CArchive ar(&file, CArchive::load);
                pView->Serialize(ar);
            }

            pView = (CDiffView *)pSplitter->GetPane(0,1);

            if(pView)
            {
                CFile   file(lpszFile2, CFile::modeRead);
                CArchive ar(&file, CArchive::load);
                pView->Serialize(ar);
            }
            //Flag as clean so that we won't get
            //prompted to save
            SetModifiedFlag(FALSE);
        }
    }
}
```

▶ **Call RunComparison from OnFileOpen**

☐ 1. Scroll to the end of **CDiffDoc::OnFileOpen**.

☐ 2. Add a call to **RunComparison** with the two files as shown:

```
RunComparison (m_File1, m_File2);
```

☐ 3. Save DiffDoc.cpp.

Notes

▶ **Set the rich edit control to read-only**

☐ 1. Open ClassWizard.

☐ 2. Add a handler for **OnInitialUpdate** to **CDiffView**.

☐ 3. Open the file DiffView.cpp.

☐ 4. Add a call to **SetReadOnly** to the end of **OnInitialUpdate** as follows:

```
GetRichEditCtrl().SetReadOnly();
```

☐ 5. Save DiffView.cpp.

▶ **Make the changes to CMainFrame**

☐ 1. Open MainFrm.h.

☐ 2. Add a public attribute method to return the splitter object as follows:

```
CSplitter * GetSplitter() { return &m_wndSplitter; }
```

☐ 3. Save MainFrm.h.

☐ 4. In the **CMainFrame::OnCreateClient** function of the file MainFrm.cpp, comment out **SetActiveView** as follows:

```
//SetActiveView((CView *)m_wndSplitter.GetPane(0,1));
```

☐ 5. Save MainFrm.cpp.

▶ **Build and test the application**

- Run your application and open a file. You will see that the file appears in the two panes of the window.

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.2\Ex03\Solution.

Notes

122 Exercise 4: Managing Drag-and-Drop Editing

Continue with the files you created in Exercise 3 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch04\Lab4.2\Ex04.

In this exercise, you will modify the behavior of **CRichEdit** so that it will open a file that is dragged from Windows Explorer to the window pane of the running application. For purposes of this exercise, you will process dropped files only if two files are dropped. Ignore any additional files beyond the first two.

▶ **Implement CMain Frame::OnDropFiles**

☐ 1. Add a handler for WM_DROPFILES to **CMainFrame** using ClassWizard.

☐ 2. Determine the number of files being dropped:

```
UINT nFileCount =
    ::DragQueryFile (hDropInfo,0xFFFFFFFF, NULL, 0);
```

☐ 3. Determine whether two or more files have been dropped:

```
if (nFileCount >= 2)
```

▶ **Edit CMainFrame::OnDropFiles to get the first two dropped files and pass them to CDiffDoc::RunComparison**

☐ 1. Get the path of the first file:

```
CString File1;

::DragQueryFile(hDropInfo,
                0,
                File1.GetBufferSetLength(_MAX_PATH),
                _MAX_PATH);
File1.ReleaseBuffer();
```

☐ 2. Repeat the process for the second file.

☐ 3. Call **CDiffDoc::RunComparison**:

```
((CDiffDoc *)GetActiveDocument())->
                RunComparison(File1, File2);
```

☐ 4. Finish drag-and-drop processing:

```
::DragFinish(hDropInfo);
```

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CMainFrame::OnDropFiles(HDROP hDropInfo)
{
    UINT nFileCount = ::DragQueryFile (hDropInfo,
                                       0xFFFFFFFF,
                                       NULL, 0);
    ASSERT(nFileCount !=0);

    // We must have at least two files for this function;
    // we will grab only the first two

    if (nFileCount >= 2)
    {
        CString File1;
        CString File2;

        ::DragQueryFile(hDropInfo,
                        0,
                        File1.GetBufferSetLength(_MAX_PATH),
                        _MAX_PATH);
        File1.ReleaseBuffer();

        ::DragQueryFile(hDropInfo,
                        1,
                        File2.GetBufferSetLength(_MAX_PATH),
                        _MAX_PATH);
        File2.ReleaseBuffer();

        ((CDiffDoc *)GetActiveDocument())->
                            RunComparison(File1, File2);

        ::DragFinish(hDropInfo);
    }
}
```

▶ **Add a handler for WM_DROPFILES to CDiffView**

- Use ClassWizard to add a handler for WM_DROPFILES to **CDiffView**.

Notes

▶ Pass the WM_DROPFILES message to CMainFrame from CDiffView

☐ 1. In **CDiffView::OnDropFiles**, get the main frame window:

```
CDiffApp::GetApp()->GetMainFrame()
```

☐ 2. Send WM_DROPFILES to it:

```
CDiffApp::GetApp()->GetMainFrame()->SendMessage(    WM_DROPFILES,
                    (WPARAM)hDropInfo );
```

☐ 3. Comment out the default **OnDropFiles** behavior so OLE embedding will not be invoked:

```
//CrichEditView::OnDropFiles(hDropInfo);
```

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CDiffView::OnDropFiles(HDROP hDropInfo)
{

    CDiffApp::GetApp()->GetMainFrame()->
                SendMessage (WM_DROPFILES,
                             (WPARAM)hDropInfo);
    // Don't call the default--we don't want OLE embedding
    //behavior
    //CRichEditView::OnDropFiles(hDropInfo);
}
```

☐ 4. In the file DiffView.cpp, include the file splitter.h and the file mainfrm.h after the file diffView.h:

```
#include "splitter.h"
#include "mainfrm.h"
```

▶ Build and test the application

☐ 1. Run the application.

☐ 2. Invoke Windows Explorer and drag any two files from Windows Explorer to the window pane of the running application.

You will see that the files appear in the two panes of the window.

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.2\Ex04\Solution.

Notes